

Миграции и начальное заполнение базы данных

Prisma Migrate, история изменений схемы и воспроизводимый seed

Результаты занятия

- ✓ Понимать назначение миграций базы данных
- ✓ Создавать и применять миграции через Prisma Migrate
- ✓ Различать `prisma migrate dev` и `prisma migrate deploy`
- ✓ Понимать назначение `shadow database`
- ✓ Контролировать историю миграций и таблицу `_prisma_migrations`
- ✓ Безопасно изменять существующую схему без ручного редактирования таблиц
- ✓ Настраивать seed-команду в `prisma.config.ts`
- ✓ Создавать воспроизводимый seed-скрипт
- ✓ Восстанавливать учебную базу из миграций и начальных данных

Учебные вопросы

- 1** **Зачем нужны миграции** Согласование Prisma Schema, SQL-схемы и истории изменений
- 2** **Создание миграций** `migrate dev`, `migration.sql` и безопасное изменение моделей
- 3** **Development и deployment** `reset`, `status` и `migrate deploy`
- 4** **Начальное заполнение** seed-скрипт и `prisma db seed`
- 5** **Рабочий процесс команды** Как воспроизводимо развернуть Internet Shop на новом компьютере



Учебный вопрос 1. Зачем нужны миграции

Схема базы данных развивается вместе с приложением

База данных меняется вместе с проектом

- На старте Product содержит только основные поля.
- Позже появляются description, sku, timestamps и новые связи.
- Изменения должны одинаково применяться на компьютерах всех разработчиков.
- Нельзя полагаться на ручное изменение таблиц через pgAdmin.
- Нужна воспроизводимая история изменения структуры базы данных.

Как выглядит изменение без миграций

- 1 Изменили `schema.prisma`** В модели `Product` появилось новое поле `description`.
- 2 Вручную изменили PostgreSQL** Один разработчик выполнил `ALTER TABLE` через SQL или pgAdmin.
- 3 Другой компьютер** На другой машине таблица осталась старой и приложение получает ошибку.
- 4 Проблема** Код и реальная структура базы данных перестали соответствовать друг другу.

Ручное изменение БД и миграции

Ручное изменение

- Изменения легко забыть
- Нет общей истории
- Сложно повторить на другой БД
- Высокий риск рассинхронизации

Prisma Migrate

- Изменения сохраняются в файлах
- Есть последовательная история
- Можно повторно применить на новой БД
- Миграции хранятся вместе с кодом

Prisma Schema → Migration → PostgreSQL

От изменения модели к работающей базе данных

1

schema.prisma

Файл со схемой
вашего проекта

```
model Product {  
  id    Int    @id  
  name  String  
  price Float  
}  
...
```

Здесь определена
структура данных
(модели и поля)

2

Изменение модели

Добавляем или изменяем
модель / поле

```
model Product {  
  id    Int    @id  
  name  String  
  price Float  
  stock Int    // новое  
}
```

Вносим изменения
в **schema.prisma**
(например, добавляем поле)

3

Prisma Migrate

Инструмент Prisma,
создающий миграцию



```
> npx prisma migrate dev
```

Сравнивает текущую схему
с предыдущей и генерирует
SQL-миграцию

4

migration.sql

SQL-скрипт миграции

```
SQL  
  
-- Add column stock  
to Product  
ALTER TABLE "Product"  
ADD COLUMN "stock"  
INTEGER;
```

Для каждого изменения
создаётся отдельный
SQL-файл миграции

5

PostgreSQL

Применяем миграцию
к базе данных



Структура БД обновляется
в соответствии с миграцией



Миграции — часть истории проекта

Каждая миграция сохраняется в папке проекта и образует историю изменений.



...



20260921_add_product_stock/

migration.sql

README.md



20260928_create_orders/

migration.sql

README.md



20261005_add_user_role/

migration.sql

README.md

...



История
проекта

Все миграции хранятся в репозитории
и позволяют отслеживать, как развивалась
структура базы данных.

Изменение схемы → новая миграция → применённые изменения → история проекта

Что такое миграция

- Миграция — это зафиксированный шаг изменения структуры базы данных, который переводит её из одного состояния схемы в другое.
- Prisma Schema описывает желаемую модель данных.
- Prisma Migrate сравнивает новое состояние с историей миграций.
- Для изменения PostgreSQL создаётся SQL-файл migration.sql.
- Применённые миграции фиксируются в базе данных.
- Файлы миграций должны храниться в системе контроля версий.

Структура каталога migrations

TEXT

```
prisma/
├── schema.prisma
├── seed.js
└── migrations/
    ├── 20260921090000_init/
    │   └── migration.sql
    └── 20260922110000_add_product_description/
        └── migration.sql
```

Что находится в migration.sql

SQL

```
ALTER TABLE "products"  
ADD COLUMN "description" TEXT;
```

Что важно

- Это обычный SQL, который изменяет PostgreSQL.
- Файл генерируется Prisma Migrate на основе изменения модели.
- SQL можно просмотреть до применения в важных проектах.
- История migration.sql позволяет воспроизвести структуру базы.

Что делает prisma migrate dev

- 1 Проверяет историю** Prisma анализирует существующие миграции и development database.
- 2 Использует shadow database** История миграций воспроизводится отдельно для обнаружения расхождений.
- 3 Создаёт новую миграцию** Изменения schema.prisma преобразуются в SQL.
- 4 Применяет миграцию** Новая миграция выполняется в development database.
- 5 Обновляет историю** Применённая миграция фиксируется в `_prisma_migrations`.

Зачем Prisma Migrate нужна shadow database

Как Prisma проверяет историю миграций и обнаруживает drift до применения новой миграции



Создание миграции в development

BASH

```
npx prisma@7 migrate dev --name add-product-description
```

Что важно

- Команда предназначена для среды разработки.
- `--name` задаёт понятное имя шага миграции.
- Prisma создаёт каталог миграции и применяет SQL.
- В Prisma ORM v7 генерацию Prisma Client выполняем отдельно.

После migrate dev генерируем Prisma Client

BASH

```
npx prisma@7 generate
```

Таблица `_prisma_migrations`

- Prisma создаёт служебную таблицу `_prisma_migrations`.
- В ней фиксируется история применённых миграций.
- По этой истории Prisma понимает состояние конкретной базы.
- Эту таблицу не используют как бизнес-таблицу приложения.
- Файлы `migrations` и записи `_prisma_migrations` должны соответствовать друг другу.



Учебный вопрос 2. Создание миграций

Изменяем Product контролируемым способом

Шаг 1. Добавляем поле description

PRISMA

```
model Product {  
  id          Int      @id @default(autoincrement())  
  name        String  
  price       Int  
  description  String?  
  categoryId   Int      @map("category_id")  
  inStock     Boolean @default(true) @map("in_stock")  
  category    Category @relation(fields: [categoryId]  
                                references: [categoryId])  
  @@map("products")  
}
```

Что важно

- String? означает nullable-поле.
- Существующие строки могут получить NULL.
- Такое изменение обычно безопаснее, чем сразу добавлять обязательное поле.
- После изменения schema.prisma создаём миграцию.

Шаг 2. Создаём и применяем миграцию

BASH

```
npx prisma@7 migrate dev --name add-product-description
```

Результат в проекте

TEXT

```
prisma/migrations/  
└─ 20260922110000_add_product_description/  
   └─ migration.sql
```

Сгенерированный SQL

SQL

```
ALTER TABLE "products"  
ADD COLUMN "description" TEXT;
```

Что важно

- Prisma создаёт SQL на основе изменения модели.
- Существующие данные остаются в таблице.
- Новый столбец допускает NULL.
- После применения Product и таблица products снова согласованы.

Миграция не означает удаление данных

- Обычный ALTER TABLE может изменить структуру без очистки таблицы.
- Риск зависит от конкретного изменения схемы.
- Добавление nullable-поля обычно не требует удаления существующих строк.
- Удаление столбца или изменение типа может привести к потере данных.
- Перед опасной миграцией нужно анализировать сгенерированный SQL.

Проверка состояния миграций

BASH

```
npx prisma@7 migrate status
```

Что важно

- Команда показывает состояние migration history.
- Можно увидеть, есть ли неприменённые миграции.
- Полезно выполнять перед deployment и при диагностике окружения.

Полный цикл изменения схемы через **migrate dev**

Как изменение модели Prisma превращается в миграцию, SQL и обновлённый Prisma Client

1 Изменение Product

schema.prisma

```
model Product {
  id    Int    @id
  name  String
  price Float
  stock Int    // новое
  ...
}
```

Меняем модель **Product**
(добавляем поле)

2 prisma migrate dev --name ...

Терминал / CLI

```
$ npx prisma
migrate dev
--name
add_product_field
```

Создаём и применяем **миграцию**
(генерируем SQL и применяем к БД)

3 Каталог migration

Папка в проекте

- prisma
- migrations
 - 20260921_add_product_field
 - migration.sql
 - README.md

История миграций сохраняется в проекте
(в отдельной папке)

4 migration.sql

SQL-файл миграции

```
SQL

ALTER TABLE "products"
ADD COLUMN "stock"
INTEGER;

-- при необходимости
-- другие изменения
```

SQL описывает изменение структуры
(создаётся Prisma)

5 Применение к PostgreSQL

Prisma Migrate



Prisma Migrate применяет SQL к БД и обновляет схему PostgreSQL

6 _prisma_migrations

Таблица в БД

_prisma_migrations	
id	text
checksum	text
finished_at	timestamptz
migration_name	text
logs	text
...	

Информация о применении миграции записывается в служебную таблицу **_prisma_migrations**

7 prisma generate

Обновление клиента

```
$ npx prisma
generate
```



Prisma Client

Генерируется обновлённый **Prisma Client**
(с новыми типами)



Итоговый результат

После изменения схемы и выполнения migrate dev у нас:



schema.prisma



schema.prisma и PostgreSQL
синхронизированы

Структура модели соответствует реальной схеме БД



prisma/migrations/

- 20260921_add_product_field/
- migration.sql
- README.md

История миграций сохранена

Все изменения хранятся в проекте и могут быть применены на других окружениях



Prisma Client

Prisma Client готов к работе

Сгенерирован с актуальными типами и отражает новую структуру модели

Вторая миграция: slug категории

PRISMA

```
model Category {  
  id      Int      @id @default(autoincrement())  
  name    String   @unique  
  slug    String?  @unique  
  products Product[]  
  
  @@map("categories")  
}
```

Что важно

- Новое поле снова сначала делаем необязательным.
- @unique создаёт ограничение уникальности.
- После изменения создаём отдельную миграцию.
- Каждое логическое изменение получает собственный шаг истории.

Какие изменения требуют особой осторожности

- Удаление столбца уничтожает данные этого столбца.
- Добавление обязательного поля проблемно, если таблица уже содержит строки.
- Изменение типа может быть несовместимо с существующими значениями.
- Добавление UNIQUE завершится ошибкой при существующих дубликатах.
- Изменения внешних ключей могут конфликтовать с существующими связями.

Безопасное добавление обязательного поля

- 1 Добавить nullable-поле** Сначала создать поле как String? и применить миграцию.
- 2 Заполнить существующие строки** Обновить данные так, чтобы NULL больше не осталось.
- 3 Сделать поле обязательным** Изменить String? на String и создать следующую миграцию.



Учебный вопрос 3. Development и deployment

Разные окружения используют разные команды

prisma migrate reset

- Команда предназначена только для development.
- Она сбрасывает development schema и повторно применяет миграции.
- Все пользовательские данные в сбрасываемой schema будут потеряны.
- После reset в Prisma ORM v7 seed запускается отдельно.
- Команда полезна, когда учебную БД нужно построить заново.

Восстановление development database

- 1 Сбросить development database** Запустить `prisma migrate reset` и подтвердить потерю данных.
- 2 Применить миграции** Prisma повторно выполняет всю историю migrations.
- 3 Сгенерировать клиент** Выполнить `prisma generate`.
- 4 Заполнить тестовыми данными** Явно выполнить `prisma db seed`.

Production и staging: migrate deploy

BASH

```
npx prisma@7 migrate deploy
```

Что важно

- Команда применяет уже существующие pending migrations.
- Она не создаёт новую миграцию из schema.prisma.
- Она не использует shadow database.
- Она не сбрасывает production database.
- Обычно запускается как часть deployment или CI/CD.

migrate dev и migrate deploy

migrate dev

- Только development
- Создаёт новые migration files
- Проверяет drift через shadow database
- Применяет миграции к dev database

migrate deploy

- Staging и production
- Не создаёт новые миграции
- Применяет только pending migrations
- Подходит для автоматического deployment

Что хранить в Git

- ✓ prisma/schema.prisma
- ✓ prisma.config.ts
- ✓ Каталог prisma/migrations
- ✓ seed-скрипт
- ✓ package.json и lock-файл
- ✓ Не хранить реальные пароли и DATABASE_URL в репозитории



Учебный вопрос 4. Начальное заполнение базы данных

Seed создаёт предсказуемые стартовые данные

Что такое seeding

- Seeding — программное начальное заполнение базы данных заранее определёнными данными.
- Seed создаёт данные, необходимые для разработки и демонстрации.
- Можно автоматически создать категории и тестовые товары.
- Новая development database получает одинаковый исходный набор данных.
- Seed должен быть воспроизводимым.
- В Prisma ORM v7 seed запускается явно командой `prisma db seed`.

Настройка seed в prisma.config.ts

TYPESCRIPT

```
import "dotenv/config";
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
    seed: "node prisma/seed.js"
  },
  datasource: {
    url: env("DATABASE_URL")
  }
});
```

Что важно

- migrations.path указывает каталог истории миграций.
- migrations.seed задаёт команду запуска seed-скрипта.
- prisma db seed выполняет именно эту команду.
- Для нашего ESM-проекта используем обычный JavaScript seed.js.

Seed: категории

JAVASCRIPT

```
import { prisma } from "../config/prisma.js";

async function main() {
  const laptops = await prisma.category.upsert({
    where: { name: "Ноутбуки" },
    update: {},
    create: { name: "Ноутбуки" }
  });

  const accessories = await prisma.category.upsert({
    where: { name: "Аксессуары" },
    update: {},
    create: { name: "Аксессуары" }
  });

  console.log({ laptops, accessories });
}
```

Что важно

- Category.name уникален, поэтому подходит для upsert.
- upsert создаёт запись, если её нет.
- Если категория уже существует, повторный запуск не создаёт дубликат.
- Это делает seed более устойчивым к повторному запуску.

Seed: тестовые товары

JAVASCRIPT

```
const laptop = await prisma.product.findFirst({
  where: { name: "Laptop Pro" }
});

if (!laptop) {
  await prisma.product.create({
    data: {
      name: "Laptop Pro",
      price: 420000,
      categoryId: laptops.id,
      inStock: true,
      description: "Учебный товар"
    }
  });
}
```

Что важно

- Product.name в нашей модели не объявлен unique.
- Поэтому сначала проверяем наличие тестового товара.
- При отсутствии создаём его через create().
- Seed можно расширить несколькими категориями и товарами.

Завершение seed-скрипта

JAVASCRIPT

```
main()
  .then(async () => {
    await prisma.$disconnect();
  })
  .catch(async (error) => {
    console.error(error);
    await prisma.$disconnect();
    process.exit(1);
  });
```

Что важно

- После успешной работы отключаем Prisma Client.
- Ошибка выводится в консоль.
- `process.exit(1)` сообщает оболочке о неуспешном выполнении seed.

Запуск начального заполнения

BASH

```
npx prisma@7 db seed
```

Migration и Seed решают разные задачи

Сначала создаём структуру базы данных, затем заполняем её начальными данными

Prisma Migrate (миграции)

Управляет структурой базы данных

Структура
Изменения
Версия

1 Определяем модель в schema.prisma

```
model Product {
  id      Int      @id @default(autoincrement())
  name    String
  price  Int
  categoryId Int
  category Category @relation(fields: [categoryId], references: [id])
}
```

2 Запускаем миграцию

```
npx prisma migrate dev --name add_product
```

3 Prisma создаёт SQL и применяет его к базе

4 Миграция сохраняется в истории проекта

Что делает миграция?

- ✓ Создаёт и изменяет таблицы
- ✓ Добавляет/удаляет столбцы
- ✓ Настраивает типы данных
- ✓ Создаёт связи (Foreign Keys)
- ✓ Создаёт индексы и ограничения



PostgreSQL

Одна база данных: сначала структура, затем данные



Структура после миграции

```
CREATE TABLE categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);

CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price INTEGER NOT NULL,
  category_id INTEGER REFERENCES
    categories(id)
);
```

Данные после seed

categories	
id	name
1	Ноутбуки
2	Смартфоны
3	Аксессуары

products			
id	name	price	category_id
1	MacBook	120000	1
2	iPhone	80000	2
3	Мышь	3000	3

Seed (начальное заполнение)

Управляет данными в базе

Данные
Справочники
Начальное состояние

1 Пишем скрипт seed (например, seed.js)

```
await prisma.category.createMany({
  data: [
    { name: "Ноутбуки" },
    { name: "Смартфоны" },
    { name: "Аксессуары" },
  ],
});
```

2 Запускаем начальное заполнение

```
npx prisma db seed
```

3 В базу добавляются справочные данные (категории, товары и т.д.)

Что делает seed?

- ✓ Заполняет справочные таблицы
- ✓ Добавляет тестовые/начальные данные
- ✓ Может создавать демо-пользователей, роли и т.д.
- ✓ Не изменяет структуру базы данных
- ✓ Можно запускать повторно (с очисткой или с проверкой существования)



Правильный порядок:

1

Миграции
(создаём структуру)



2

Seed
(заполняем данные)



3

Готовая база данных
(структура + начальные данные)



Миграции и seed дополняют друг друга:
сначала структура, затем данные.

Migration vs Seed

Migration

- Изменяет структуру базы
- Создаёт таблицы и столбцы
- Добавляет ограничения и индексы
- Хранит историю schema changes

Seed

- Создаёт начальные данные
- Добавляет категории и тестовые записи
- Не заменяет миграции
- Запускается после готовности структуры



Учебный вопрос 5. Рабочий процесс команды

Новый компьютер должен получить ту же схему и стартовые данные

Развёртывание проекта на новом компьютере

- 1 Установить зависимости** Выполнить `npm install` и настроить `.env`.
- 2 Применить миграции** В `development` выполнить `prisma migrate dev` для синхронизации локальной базы с `migration history`.
- 3 Сгенерировать Prisma Client** Выполнить `prisma generate`.
- 4 Запустить seed** Выполнить `prisma db seed` для создания стартовых данных.
- 5 Запустить API** Проверить REST endpoints Internet Shop.

Типичная последовательность команд

BASH

```
npm install  
npx prisma@7 migrate dev  
npx prisma@7 generate  
npx prisma@7 db seed  
npm start
```

Типичные ошибки

● Изменили БД вручную

Появляется schema drift: реальная структура расходится с migration history.

● Удалили migration file

История проекта больше не соответствует применённым миграциям.

● Использовали migrate dev в production

Development-команда не предназначена для production deployment.

● Ожидали автоматический seed

В Prisma ORM v7 seed запускается явно через prisma db seed.

● Seed создаёт дубликаты

Нужно использовать уникальные ключи, upsert или предварительную проверку.

1 schema.prisma

Описываем модели и связи



prisma/schema.prisma

```
model User {
  id      Int    @id @default(autoincrement())
  name    String
  email   String @unique
  orders  Order[]
}

model Product {
  id          Int    @id @default(autoincrement())
  name        String
  price       Int
  categoryId  Int
  category    Category @relation(fields: [categoryId], references: [id])
}
```

2 Migrations

Создаём и применяем миграции

```
npm prisma migrate dev
--name init
```

prisma/migrations/

- 20250917093000_init/
- migration.sql
- README.md

Миграция содержит SQL изменения и сохраняется в истории проекта.

3 PostgreSQL

Структура базы данных



Созданы таблицы:

users
products
categories
order_items
orders
_prisma_migrations

4 prisma generate

Генерируем Prisma Client

```
npm prisma generate
```

node_modules/
@prisma/client/

- Типобезопасный клиент
- Автодополнение в IDE
- Типы из вашей схемы
- Готов к использованию в приложении

7 Express REST API

Работает с готовой базой данных



express JS

GET	/api/products
GET	/api/products/:id
POST	/api/orders
GET	/api/orders/:id
...	

API использует Prisma Client для работы с данными в PostgreSQL.

5 seed.js

Подготовка начальных данных



prisma/seed.js

```
async function main() {
  await prisma.category.createMany({
    data: [
      { name: "Ноутбуки" },
      { name: "Смартфоны" },
      { name: "Аксессуары" },
    ],
  });
  // ... добавление товаров, пользователей
}
main();
```

6 prisma db seed

Заполняем базу начальными данными

```
npm prisma db seed
> Seed completed successfully!
```



PostgreSQL

База данных с данными

Данные в таблицах:

users	(например, администратор)
categories	(3 категории)
products	(начальные товары)
orders	(примерные заказы)
order_items	(позиции заказов)

Готовая база данных со структурой и начальными данными используется в приложении

1 schema.prisma

Описываем модели

2 migrate dev

Создаём миграции

3 PostgreSQL

Применяем структуру

4 prisma generate

Генерируем клиент

5 seed.js

Пишем начальные данные

6 db seed

Заполняем базу

7 Express API

Работает с данными

Контрольные вопросы

1. Какую проблему решают миграции?
2. Что хранится в `migration.sql`?
3. Для чего используется `shadow database`?
4. Чем `migrate dev` отличается от `migrate deploy`?
5. Почему `migrate dev` нельзя использовать как `production`-команду?
6. Что делает `migrate reset`?

7. Для чего нужна таблица `_prisma_migrations`?
8. Что такое `seed`?
9. Где в Prisma ORM v7 задаётся команда `seed`?
10. Почему `seed` желательно делать повторяемым?
11. В каком порядке на новой машине применяются `migrations`, `generate` и `seed`?
12. Почему `migration files` должны храниться в Git?

Итоги занятия

1

Prisma Migrate хранит воспроизводимую историю изменения структуры базы.

2

`migrate dev` создаёт и применяет миграции в `development`.

3

`Shadow database` помогает обнаруживать расхождения `migration history` и `development schema`.

4

Каждая миграция содержит SQL-файл изменения базы.

5

Опасные `schema changes` необходимо проверять до применения.

6

`migrate deploy` применяет существующие `pending migrations` в `staging` и `production`.

7

`migrate reset` предназначен только для `development` и удаляет данные.

8

`Seed` создаёт предсказуемый набор начальных данных.

9

В Prisma ORM v7 `seed` запускается явно через `prisma db seed`.

1

0

Миграции отвечают за структуру, `seed` — за стартовые данные.

1

1

Новый разработчик может воспроизвести `Internet Shop` из `migration history` и `seed`-скрипта.